



RAJALAKSHMI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ACADEMIC
YEAR 2026 - 2027**

SEMESTER V

AL23531 - NATURAL LANGUAGE PROCESSING

MINI PROJECT REPORT

REGISTER NUMBER	2117240030146
NAME	SRIRAM J
PROJECT TITLE	Probabilistic CYK Algorithm for Finding Most Likely Parse Trees
DATE OF SUBMISSION	
FACULTY IN-CHARGE	MR. SHREE MAKESH N

Signature of Faculty In-charge

Signature of Hod

Internal Examiner

External Examiner

1. Project Title

Probabilistic CYK Algorithm for Finding Most Likely Parse Trees

2. Introduction

Natural Language Processing involves computational techniques for understanding and analyzing human language. Syntactic parsing is an important NLP task because it identifies the hierarchical grammatical structure of a sentence. A parse tree represents how words combine into phrases and larger constituents.

The CYK algorithm is a dynamic-programming algorithm for context-free grammars in Chomsky Normal Form. It systematically evaluates spans of a sentence and determines which grammar symbols can generate each span. A probabilistic version associates probabilities with grammar productions and chooses the derivation with the greatest probability.

This project implements a Probabilistic CYK parser using a small Probabilistic Context-Free Grammar (PCFG). The application accepts an input sentence, fills a probability chart, stores Viterbi-style back-pointers, and reconstructs the most likely parse tree. A browser-based frontend presents the results interactively.

3. Problem Statement

Given an input sentence and a probabilistic context-free grammar, identify a valid syntactic parse and select the parse tree having the highest probability when multiple derivations are possible. The project should make the probabilistic decision process visible and provide an accessible interface for testing sentences.

- Represent a grammar using weighted production rules.
- Break a sentence into tokens and initialize lexical probabilities.
- Evaluate every span and split using CKY dynamic programming.
- Retain the highest-probability derivation for each non-terminal.
- Reconstruct and display the most likely parse tree.
- Provide backend and frontend components for demonstration.

4. Aim / Goal

The main aim is to design and implement a Probabilistic CYK algorithm that analyzes an input sentence using a Probabilistic Context-Free Grammar and identifies the most likely syntactic parse tree.

5. Objectives

1. Understand Context-Free Grammar and Probabilistic Context-Free Grammar concepts.
2. Implement CKY/CYK dynamic programming for binary grammar productions.
3. Assign probabilities to lexical and grammatical productions.
4. Store the best probability for every non-terminal and sentence span.
5. Use back-pointers to reconstruct the Viterbi-style parse.
6. Build a Flask backend to expose the parser through a REST API.
7. Develop an HTML/CSS/JavaScript frontend for interactive testing.
8. Display the parse tree, bracketed representation and CKY chart.
9. Visualize selected constituent probabilities and algorithm statistics.
10. Document results, limitations and future enhancements.

6. Linguistic Background

A Context-Free Grammar uses non-terminals, terminals, production rules and a start symbol. A PCFG extends this framework by associating probabilities with productions. In probabilistic parsing, the probability of a derivation is calculated from the probabilities of the rules used in that derivation.

- Lexical rules connect words to grammatical categories.
- Binary rules combine two child constituents into a parent constituent.
- CKY evaluates smaller spans before larger spans.
- Viterbi back-pointers retain the split and child symbols of the best derivation.
- The highest-probability complete S constituent represents the selected parse.

7. Dataset / Input Description

Unlike a conventional machine-learning project, this implementation does not require a numerical dataset. The main input is a sentence together with a PCFG. A small controlled vocabulary is supplied so that the algorithm can be demonstrated reproducibly.

VARIABLE / ITEM	TYPE	DESCRIPTION
Sentence	Text	Input sentence to be parsed
Token	Text	Individual word obtained after tokenization
Non-terminal	Categorical	Grammar symbol such as S, NP, VP, Det or N
Production	Grammar rule	Binary or lexical grammar production
Rule probability	Numerical	Probability associated with a production
Chart span	Numerical pair	Start and end token positions
Cell probability	Numerical	Best probability for a non-terminal in a span
Back-pointer	Structured	Information required to reconstruct the best tree

8. PCFG Rules Used

RULE	PROBABILITY
$S \rightarrow NP VP$	1.00
$NP \rightarrow Det N$	0.55
$NP \rightarrow Det AdjN$	0.25
$NP \rightarrow N N$	0.20
$AdjN \rightarrow Adj N$	1.00
$VP \rightarrow V NP$	0.60
$VP \rightarrow V Adv$	0.40
$Det \rightarrow the$	0.60
$Det \rightarrow a$	0.40
$N \rightarrow boy / girl / dog / cat$	0.25 each
$Adj \rightarrow big / small$	0.50 each
$V \rightarrow sees / likes$	0.50 each
$Adv \rightarrow quickly$	1.00

The grammar is deliberately small for laboratory demonstration. Its probabilities are project-level values and should not be interpreted as probabilities learned from a large English treebank.

9. Data Preprocessing

9.1 Tokenization

The input sentence is converted to lowercase and split on whitespace to obtain an ordered list of tokens.

9.2 Vocabulary Validation

Each token is checked against the project lexicon. An informative error is returned if an unknown word is supplied.

9.3 Grammar Preparation

The demonstration grammar uses lexical productions and binary productions so that the standard CKY recurrence can be applied.

9.4 Probability Handling

The parser multiplies child probabilities with the probability of the grammar production. When competing candidates generate the same constituent, only the maximum is retained.

10. System Architecture

Probabilistic CYK Parsing Architecture

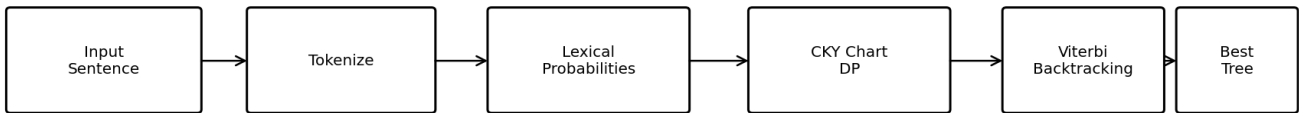


Figure 1. Probabilistic CYK system architecture.

STAGE	OPERATION	OUTPUT
Input	Read sentence	Raw sentence
Tokenization	Split sentence	Token sequence
Lexical initialization	Apply word rules	Base chart cells
CKY DP	Evaluate spans and splits	Best constituent probabilities
Backtracking	Follow stored pointers	Most likely parse
Presentation	Render results	Tree, chart and metrics

11. Probabilistic CYK Algorithm

INPUT: sentence $W = w_1 \dots w_n$ and PCFG G
OUTPUT: highest-probability parse tree

1. Tokenize W .
2. Initialize $\text{chart}[i][i+1]$ with lexical probabilities.
3. For $\text{span} = 2 \dots n$:
 For each start i :
 $j = i + \text{span}$
 For each split k :
 For each rule $A \rightarrow B C$:
 $\text{candidate} =$
 $\text{chart}[i][k][B] * \text{chart}[k][j][C] * P(A \rightarrow B C)$
 Keep candidate if it is greater than the current probability for A .
 Store a back-pointer.
4. Check $\text{chart}[0][n]$ for start symbol S .
5. Follow back-pointers recursively from S .
6. Return the tree and its probability.

12. Mathematical Formulation

For a binary production $A \rightarrow B C$ and split point k :

$$P(A, i, j) = \max_k [P(B, i, k) \times P(C, k, j) \times P(A \rightarrow B C)]$$

The max operation is the Viterbi step. It ensures that for each constituent the parser retains the derivation with the greatest probability instead of summing all possible derivations.

13. Worked Example

Input: the boy sees the dog

CONSTITUENT	CALCULATION	VALUE
NP(the boy)	$0.60 \times 0.25 \times 0.55$	0.082500
NP(the dog)	$0.60 \times 0.25 \times 0.55$	0.082500
VP(sees the dog)	$0.50 \times 0.0825 \times 0.60$	0.024750
S(entire sentence)	$0.0825 \times 0.02475 \times 1.00$	0.002041875

14. Backend Implementation

The backend is implemented in Python using Flask. The parser is separated into its own module so the algorithm can be tested independently from the web interface.

14.1 Flask API

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from pcfg_parser import PCFGParser

app = Flask(__name__)
CORS(app)
parser = PCFGParser()

@app.post("/api/parse")
def parse_sentence():
    body = request.get_json(silent=True) or {}
    sentence = body.get("sentence", "")
    return jsonify(parser.parse(sentence))

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000, debug=True)
```

14.2 Core CKY Logic

```
for span in range(2, n + 1):
    for i in range(n - span + 1):
        j = i + span
        for k in range(i + 1, j):
            for lhs, rhs1, rhs2, rule_p in binary_rules:
                if rhs1 in chart[i][k] and rhs2 in chart[k][j]:
                    candidate = (
                        chart[i][k][rhs1]["prob"] *
                        chart[k][j][rhs2]["prob"] *
                        rule_p
                    )
                    if candidate > best_probability:
                        chart[i][j][lhs] = {
                            "prob": candidate,
                            "back": (k, rhs1, rhs2)
                        }
```

14.3 API Endpoints

ENDPOINT	METHOD	PURPOSE
/	GET	Serves the web interface
/api/grammar	GET	Returns grammar rules
/api/parse	POST	Parses the submitted sentence

15. Frontend Implementation

The frontend is developed using HTML, CSS and JavaScript. It provides a simple laboratory-style interface where the user can enter a sentence and immediately inspect the probabilistic parsing result.

15.1 Main Interface Components

- Sentence input box and Parse Sentence button.
- Example sentence buttons for quick testing.
- KPI cards showing probability, token count, chart cells and candidate operations.
- Visual parse-tree area.
- Bracketed tree output area.
- CKY probability chart.
- CKY chart table containing spans and probabilities.
- PCFG rule listing and algorithm explanation.

15.2 JavaScript API Request

```
const response = await fetch("/api/parse", {  
  method: "POST",  
  headers: {"Content-Type": "application/json"},  
  body: JSON.stringify({sentence})  
});  
const data = await response.json();
```

15.3 Sample Frontend Output

Most Likely Parse Tree: the boy sees the dog

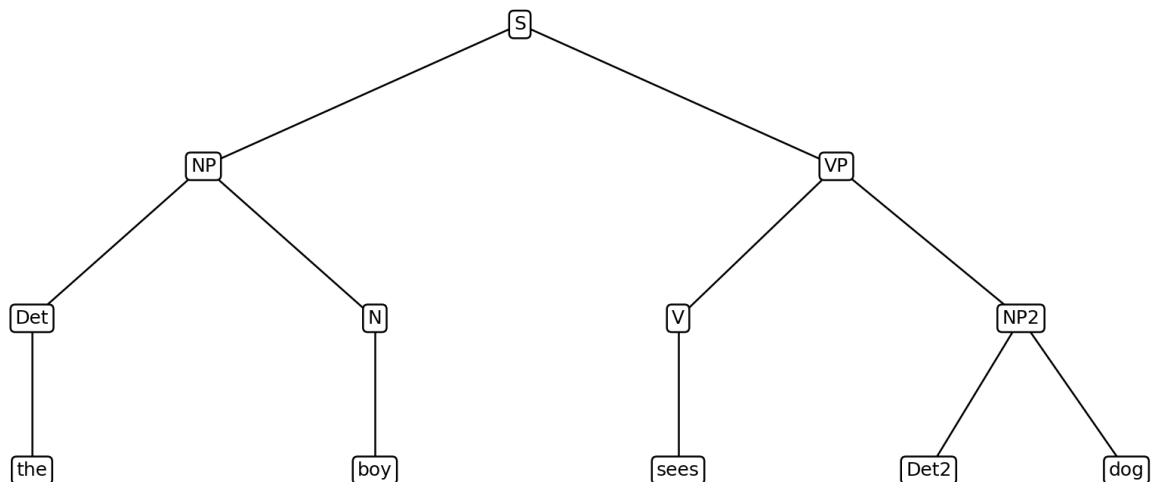


Figure 2. Most likely parse tree rendered by the application.

(S (NP (Det the) (N boy)) (VP (V sees) (NP (Det the) (N dog)))))

Most likely probability: 0.002041875

16. Data Visualization / Results

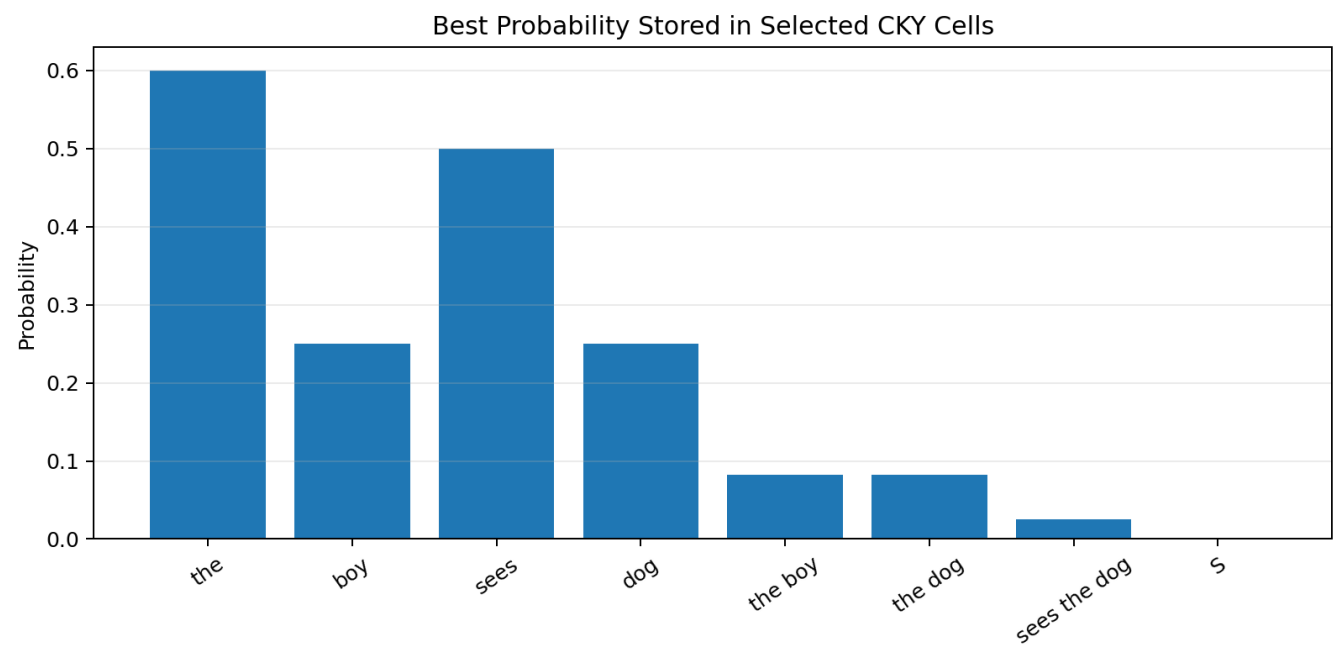


Figure 3. Selected CKY cell probabilities for the demonstration sentence.

16.1 CKY Chart Statistics

MEASURE	VALUE
Sentence length	5 tokens
Number of possible spans	15
Start symbol	S
Complete parse found	Yes
Most likely probability	0.002041875
Probability percentage	0.2041875%

16.2 Complexity

For n tokens, CKY evaluates $O(n^2)$ spans and up to $O(n)$ split points for each span. With a grammar containing $|G|$ applicable binary rules, the broad time complexity is $O(n^3 \times |G|)$, while chart storage is approximately $O(n^2 \times |N|)$, where $|N|$ is the number of non-terminals.

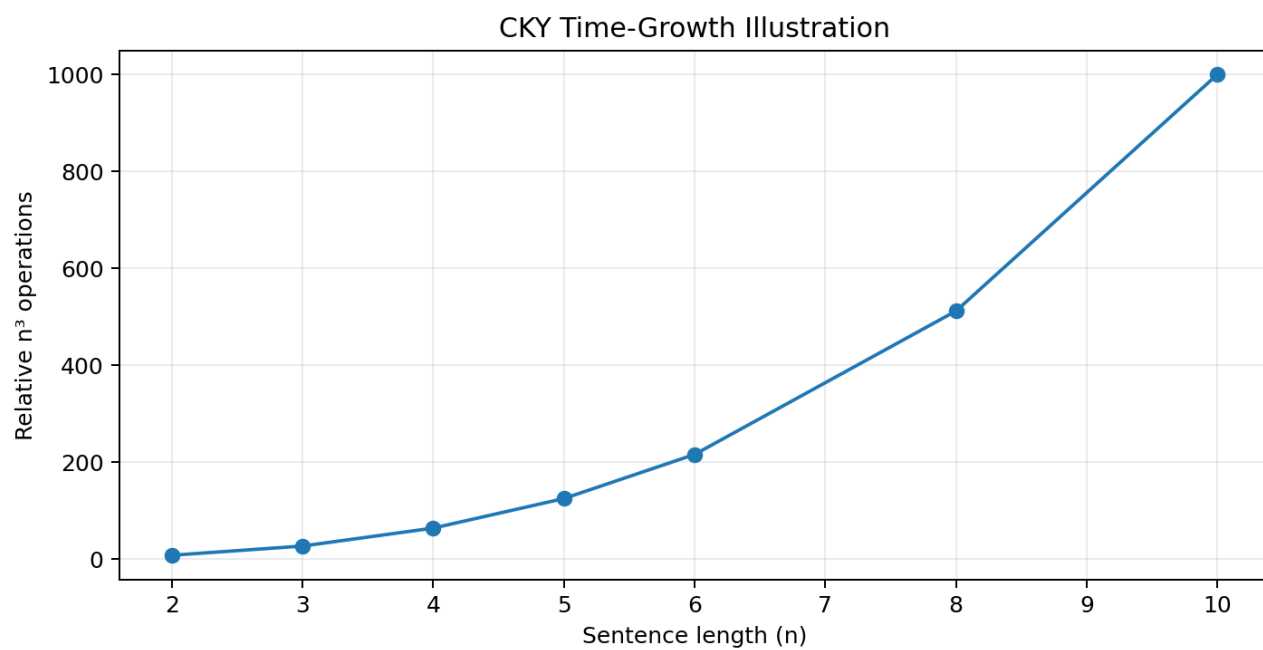


Figure 4. Illustration of cubic growth in the sentence-length component of CKY computation.

17. Testing / Validation

TEST CASE	INPUT	EXPECTED RESULT
TC01	the boy sees the dog	Valid S parse
TC02	a girl likes a cat	Valid S parse
TC03	the big dog sees a small cat	Valid S parse
TC04	the boy unknownword	Unknown-word error
TC05		Input validation error

18. Advantages

- Finds the most likely parse instead of an arbitrary valid parse.
- Dynamic programming avoids repeated subproblem computation.
- Back-pointers make parse-tree reconstruction systematic.
- Probabilities provide a quantitative ranking of derivations.
- Frontend and backend separation makes the project easy to demonstrate.
- Internal CKY chart output improves algorithm transparency.

19. Limitations

- The demonstration grammar covers a limited subset of English.
- Grammar probabilities are manually specified for academic demonstration.
- Unknown-word handling is simple and lexicon-dependent.
- Large grammars and long sentences increase computational cost.
- The project does not train a PCFG automatically from a treebank.
- Standard corpus-level parsing metrics are not included.

20. Future Enhancements

- Train grammar probabilities from Penn Treebank or another annotated treebank.
- Add automatic POS tagging and grammar conversion to Chomsky Normal Form.
- Display multiple candidate parse trees and their probabilities.
- Animate chart filling and split selection step by step.
- Support larger grammars and unknown-word smoothing.
- Compare deterministic CKY with probabilistic CKY.
- Evaluate against gold parse trees using labeled precision, recall and F1.
- Add interactive grammar upload and model management.

21. Results and Discussion

The implementation demonstrates that a probabilistic parser can combine lexical evidence and grammar probabilities to select a single best syntactic structure. For the demonstration sentence, the system successfully constructs an S-rooted parse and returns the probability and derivation in both graphical and bracketed forms.

22. Conclusion

This project demonstrates the architecture and implementation of the Probabilistic CYK algorithm for finding the most likely parse tree. By combining PCFG rules, dynamic programming and Viterbi-style backtracking, the system provides an interpretable method for probabilistic syntactic parsing. The full-stack implementation makes the algorithm accessible through a browser and presents its internal chart and outputs clearly.

23. Tools and Technologies Used

TOOL / TECHNOLOGY	PURPOSE
Python	Parser and backend implementation
Flask	REST API and local server
HTML5	Frontend structure
CSS3	Frontend design
JavaScript	API communication and visualization
PCFG	Probabilistic grammar
CKY/CYK	Dynamic-programming parsing
Matplotlib	Report graphs

24. References

- Jurafsky, D. and Martin, J. H., Speech and Language Processing.
- Natural Language Processing course materials on CFG, PCFG and CKY parsing.
- Python Documentation.
- Flask Documentation.
- Academic references on probabilistic parsing and context-free grammars.
- The uploaded NLP mini-project report was used as the structural and presentation reference for this report.